Part-A

PROGRAM 1

Write a Python program to find roots of a quadratic equation.

```
import math
print("Quadratic Equation: ax^2 + bx + c = 0")
a = float(input("Enter coefficient a: "))
b = float(input("Enter coefficient b: "))
c = float(input("Enter coefficient c: "))
if a == 0:
  print("It is not a quadratic equation")
else:
    d = b * b - 4 * a * c

    print("Discriminant =", d)

    if d > 0:
        root1 = (-b + math.sqrt(d)) / (2 * a)
        root2 = (-b - math.sqrt(d)) / (2 * a)
        print("Roots are real and distinct")
        print("Root 1 =", root1)
        print("Root 2 =", root2)

    elif d == 0:
        root = -b / (2 * a)
        print("Roots are real and equal")
        print("Root =", root)

    else:
        real = -b / (2 * a)
        imag = math.sqrt(-d) / (2 * a)
        print("Roots are complex")
        print("Root 1 =", real, "+", imag, "i")
        print("Root 2 =", real, "-", imag, "i")
```

Output 1:

Quadratic Equation: ax^2 + bx + c = 0
Enter coefficient a: 2
Enter coefficient b: 3
Enter coefficient c: 4
Discriminant = -23.0
Roots are complex
Root 1 = -0.75 + 1.1989578808281798 i
Root 2 = -0.75 - 1.1989578808281798 i

Output 2:

Quadratic Equation: ax^2 + bx + c = 0
Enter coefficient a: 3
Enter coefficient b: 4
Enter coefficient c: 6
Discriminant = -56.0
Roots are complex
Root 1 = -0.6666666666666666 + 1.247219128924647 i
Root 2 = -0.6666666666666666 - 1.247219128924647 i

PROGRAM 2

Write a Python program to find sum of n natural numbers.

```
print("Sum of First n Natural Numbers")
n = int(input("Enter value of n: "))
sum = 0
for i in range(1, n + 1):
    sum = sum + i
print("Sum =", sum)
```

Output 1:

```
Sum of First n Natural Numbers
Enter value of n: 6
Sum = 21
```

Output 2:

```
Sum of First n Natural Numbers
Enter value of n: 7
Sum = 28
```

PROGRAM 3

Program Name: Write a Python program to print multiplication table.

```
print("Multiplication Table")
n = int(input("Enter a number: "))
for i in range(1, 11):
  print(n, "x", i, "=", n * i)
```

Output 1:

```
Multiplication Table
Enter a number: 6
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
6 x 10 = 60
```

Output 2:

```
Multiplication Table
Enter a number: 10
10 x 1 = 10
10 x 2 = 20
10 x 3 = 30
10 x 4 = 40
10 x 5 = 50
10 x 6 = 60
10 x 7 = 70
10 x 8 = 80
10 x 9 = 90
10 x 10 = 100
```

PROGRAM 4

Program Name: Write a Python program to check whether a number is prime or not.

```
num = int(input("Enter the number to check if it is prime or not: "))
# If given number is greater than 1
if num > 1:
    # Iterate from 2 to n // 2
    for i in range(2, (num//2)+1):
        # If num is divisible by any number between
        # 2 and n / 2, it is not prime
        if (num % i) == 0:
            print(num, "is not a prime number")
            break
    else:
        print(num, "is a prime number")
else:
    print(num, "is not a prime number")
```

Output:

```
Enter the number to check if it is prime or not: 6
6 is not a prime number

Enter the number to check if it is prime or not: 13
13 is a prime number
```

PROGRAM 5

Program Name: Write a Python program to find factorial of a number using recursive function.

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

n = int(input("Enter a number: "))
print("Factorial =", factorial(n))
```

Output 1:

Enter a number: 6
Factorial = 720

Output 2:

Enter a number: 5
Factorial = 120

PROGRAM 6

Program Name: Write a Python program to find square of a number using exception handling until the number is valid.

```
print("Square of a Number Using Exception Handling")
while True:
    try:
        n = int(input("Enter a number: "))
        square = n * n
        print("Square =", square)
        break
    except ValueError:
        print("Invalid input. Please enter a valid integer.")
```

Output 1:

Square of a Number Using Exception Handling
Enter a number: 4
Square = 16

Output 2:

Square of a Number Using Exception Handling
Enter a number: 9
Square = 81

PROGRAM 7

Program Name: Write a Python program to read a sentence and find the number of vowels.

```
print("Count Number of Vowels")
sentence = input("Enter a sentence: ")
count = 0
for ch in sentence:
    if ch in 'aeiouAEIOU':
        count += 1
print("Number of vowels =", count)
```

Output 1:
Count Number of Vowels
Enter a sentence: Mahesh
Number of vowels = 2

Output 2:
Count Number of Vowels
Enter a sentence: GFGC Navanagar
Number of vowels = 4

PROGRAM 8

Program Name: Write a Python program using lists to store book titles allowing users to add books, search for books, and display all books.

```
print("Book List Operations")
books = []
for i in range(3):
    title = input("Enter book title: ")
    books.append(title)
print("Book List:")
for b in books:
    print(b)
search = input("Enter book title to search: ")
if search in books:
    print("Book found")
else:
    print("Book not found")
```

Output 1:

```
Book List Operations
Enter book title: c
Enter book title: c++
Enter book title: java
Book List:
c
c++
java
Enter book title to search: c
Book found
```

Output 2:

```
Book List Operations
Enter book title: kannada
Enter book title: english
Enter book title: web technology
Book List:
kannada
english
web technology
Enter book title to search: python
book not found
```

PROGRAM 9

Program Name: Write a Python program to read a sentence and count frequency of words.

Program:

```
print("Word Frequency Count")
sentence = input("Enter a sentence: ")
words = sentence.split()
for word in set(words):
  print(word, ":", words.count(word))
```

Output 1:

```
Word Frequency Count
Enter a sentence: mahesh mahesh manu tanu
tanu : 1
mahesh : 2
manu : 1
```

Output 2:

```
Word Frequency Count
Enter a sentence: GFGC GFGC Navanagar Bagalkot
Bagalkot : 1
GFGC : 2
Navanagar : 1
```

PROGRAM 10

Program Name: Write a Python program to implement stack operations.

```
print("Stack Operations")
stack = []
stack.append(10)
stack.append(20)
stack.append(30)
print("Stack after push:", stack)
item = stack.pop()
print("Deleted element:", item)
print("Stack after pop:", stack)
```

Output 1:

```
Stack Operations
Stack after push: [10, 20, 30]
Deleted element: 30
Stack after pop: [10, 20]
```

Output 2:

```
Stack Operations
Stack after push: [10, 20, 30]
Deleted element: 30
Stack after pop: [10, 20]
```

Part B

PROGRAM 11

Program Name: Write a Python program to read five subject names of BCA and five subject names of BSC and find (a) all subject names of both (b) common subjects (c) subjects only for BCA using set operations.

```
print("Set Operations on Subject Names")
bca = set()
bsc = set()

print("Enter 5 BCA subjects:")
for i in range(5):
    sub = input()
    bca.add(sub)

print("Enter 5 BSC subjects:")
for i in range(5):
    sub = input()
    bsc.add(sub)

print("All subjects:", bca | bsc)
print("Common subjects:", bca & bsc)
print("Subjects only for BCA:", bca - bsc)
```

Output 1:

```
Set Operations on Subject Names
Enter 5 BCA subjects:
c
java
DBMS
C#
DMS
Enter 5 BSC subjects:
Web
ML
AI
Python
PHP
All subjects: {'DMS', 'PHP'}
Common subjects: set()
Subjects only for BCA: {'DMS'}
```

Output 2:

```
Set Operations on Subject Names
Enter 5 BCA subjects:
c
web
html
php
python
Enter 5 BSC subjects:
c
dbms
dms
java
project
All subjects: {'c', 'python', 'dbms', 'php', 'dms', 'web', 'java', 'project', 'html'}
Common subjects: {'c'}
Subjects only for BCA: {'html', 'python', 'php', 'web'}
```

PROGRAM 12
Program Name: Write a Python program to create a 2D NumPy array for customer sales data, compute total spending per customer, find the most purchased product category, and identify the highest spender.

```
import numpy as np

print("Customer Sales Analysis")

sales = np.array([[200, 300, 400],
                  [150, 250, 350],
                  [300, 200, 500]])

print("Sales Data:")
print(sales)

total = sales.sum(axis=1)
print("Total spending per customer:", total)

category = sales.sum(axis=0)
print("Most purchased category index:", np.argmax(category))

print("Highest spender index:", np.argmax(total))
```

Output 1:

```
Customer Sales Analysis
Sales Data:
[[200 300 400]
 [150 250 350]
 [300 200 500]]
Total spending per customer: [ 900  750 1000]
Most purchased category index: 2
Highest spender index: 2
```

Output 2:

```
Customer Sales Analysis
Sales Data:
[[200 300 400]
 [150 250 350]
 [300 200 500]]
Total spending per customer: [ 900  750 1000]
Most purchased category index: 2
Highest spender index: 2
```

PROGRAM 13
Program Name: Write a Python program to load an employee dataset (CSV), filter employees who joined after 2020, calculate the average salary per department, and find highest and lowest paying departments.

```
import pandas as pd

# Load CSV
df = pd.read_csv('employees.csv')

# Clean column names
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

print("Columns:", df.columns)  # Debug

# Convert date column
df['joining_date'] = pd.to_datetime(df['joining_date'], errors='coerce')

# Filter after 2020
filtered_df = df[df['joining_date'].dt.year > 2020]

# Group by department
avg_salary = filtered_df.groupby('department')['salary'].mean()

# Results
print(filtered_df)
print("\nAverage Salary:\n", avg_salary)
print("\nHighest:", avg_salary.idxmax())
print("Lowest:", avg_salary.idxmin())
```

Create CSV file in Excel

File Name: employees.csv

employee_id	name	department	salary	joining_date
1	Mahesh	IT	60000	2021/05/10
2	Vinod	HR	50000	2020/10/02
3	Sham	Finance	30000	2019/02/03
4	Vijay	Engineer	50000	2022/02/20
5	Pallavi	HR	51000	2022/05/25
6	Savita	Excecutive	51000	2021/06/20
7	Praveen	Sales	25000	2019/04/21
8	Raja	IT	56000	2020/04/25

Output 1:

Columns: Index(['employee_id', 'name', 'department', 'salary', 'joining_date'], dtype='str')

employee_id	name	department	salary	joining_date
1	Mahesh	IT	60000	2021-05-10
4	Vijay	Engineer	50000	2022-02-20
5	allavi	HR	51000	2022-05-25
6	Savita	Excecutive	51000	2021-06-20

Department: Average Salary
Engineer 50000.0
Excecutive 51000.0
HR 51000.0
IT 60000.0

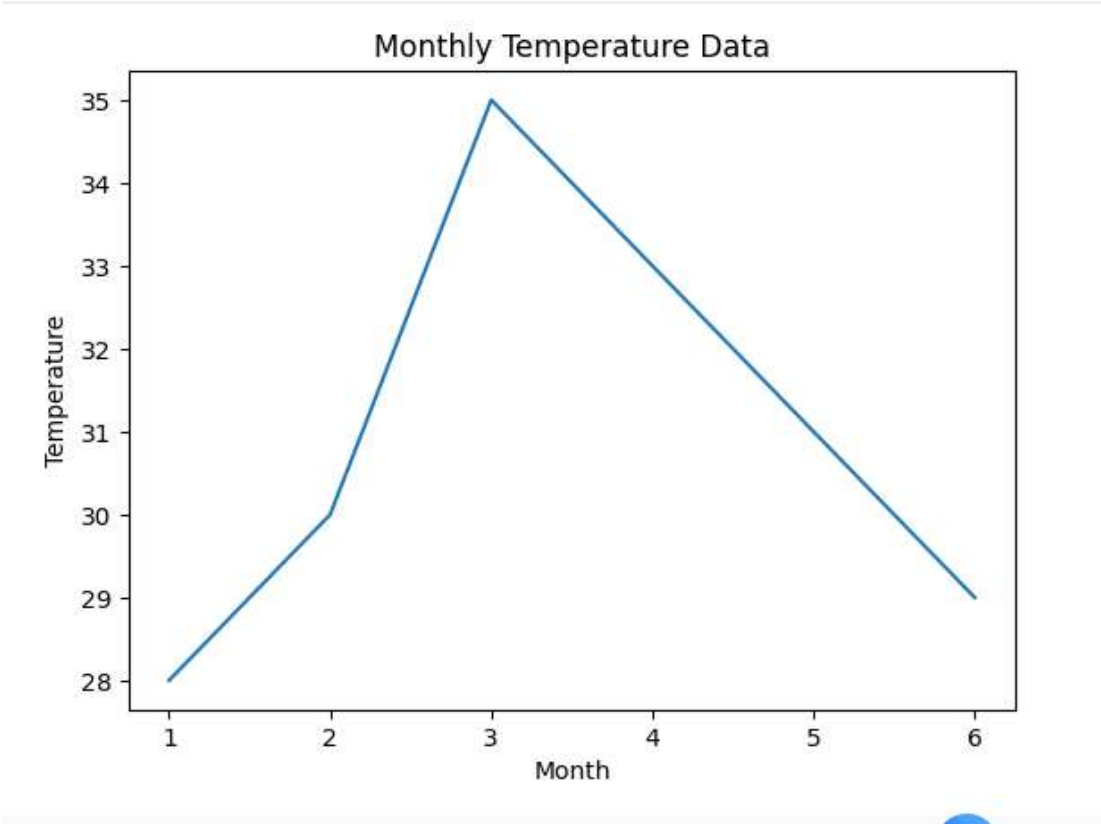
Highest: IT
Lowest: Engineer

PROGRAM 14

Program Name: Write a Python program to plot a line chart for monthly temperature data labelling axes, adding title, and customizing colors.

```
import matplotlib.pyplot as plt
months = [1, 2, 3, 4, 5, 6]
temperature = [28, 30, 35, 33, 31, 29]

plt.plot(months, temperature)

plt.xlabel("Month")
plt.ylabel("Temperature")
plt.title("Monthly Temperature Data")

plt.show()
```

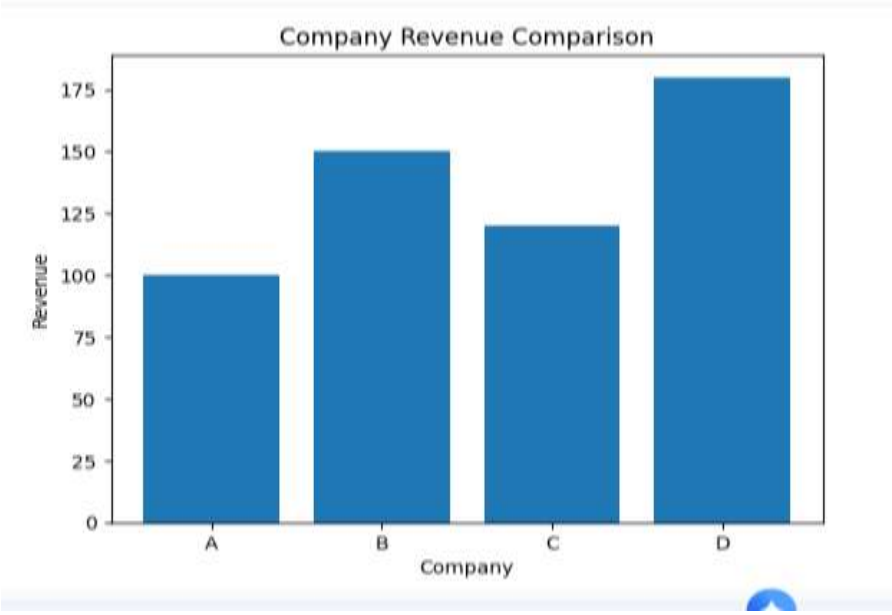
Output 1:



PROGRAM 15
Program Name: Write a Python program to load a dataset, extract relevant columns, and create a bar chart to compare total revenue of different companies

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
          'Company': ['A', 'B', 'C', 'D'],
          'Revenue': [100, 150, 120, 180]
        }

df = pd.DataFrame(data)
plt.bar(df['Company'], df['Revenue'])
plt.xlabel("Company")
plt.ylabel("Revenue")
plt.title("Company Revenue Comparison")
plt.show()
```

Output 1:



PROGRAM 16
Program Name: Write a Python Flask App to create customer HTML register form template and store data using POST method into MySQL table customer.

Steps to install packages in cmd
pip install flask
pip install mysql-connector-python
pip install mysql

Create Folder name like flask app:
```
Flask_app/
│── app.py
│── templates/
   └── register.html
```

To run the program
Cmd –python app.py
Copy web ip address
Paste in browser
Enter input

File Name: app.py

File Name: app.py

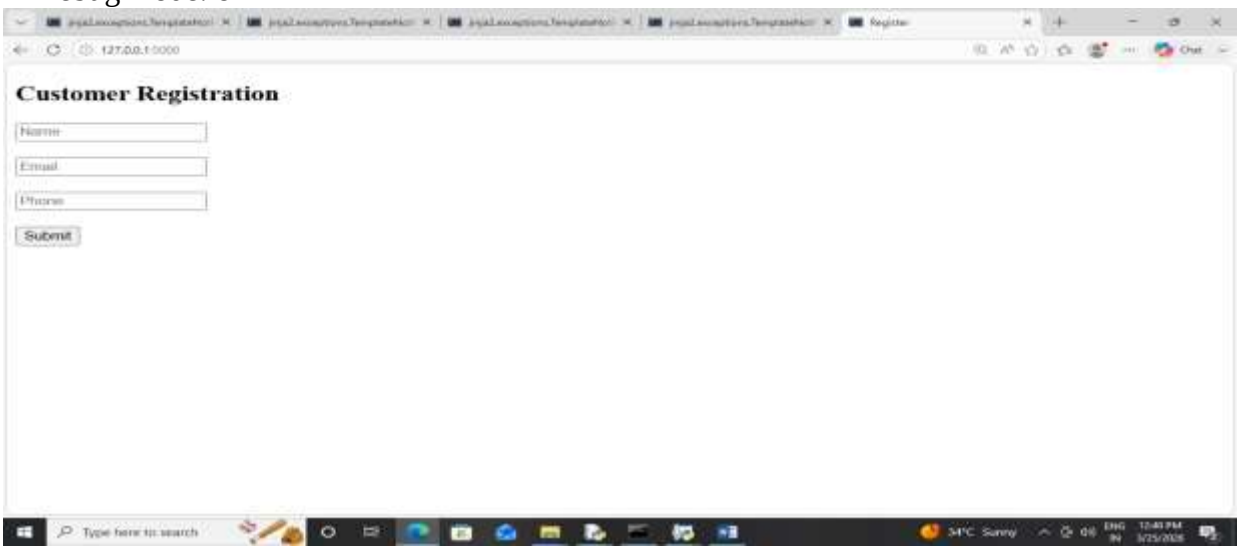
```
from flask import Flask, render_template, request
import sqlite3
app = Flask(__name__)
# Function to initialize database
definit_db():
with sqlite3.connect('database.db') as conn:
cursor = conn.cursor()
cursor.execute('''
        CREATE TABLE IF NOT EXISTS customer (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,
email TEXT,
phone TEXT
        )
    ''')
conn.commit()

# Home route
@app.route('/')
def home():
return render_template('register.html')

# Register route
@app.route('/register', methods=['POST'])
def register():
name = request.form.get('name')
email = request.form.get('email')
phone = request.form.get('phone')

if not name or not email or not phone:
return "All fields are required!", 400

with sqlite3.connect('database.db') as conn:
cursor = conn.cursor()
cursor.execute(
        "INSERT INTO customer (name, email, phone) VALUES (?, ?, ?)",
        (name, email, phone)
    )
conn.commit()
return "Customer Registered Successfully!"
# Run app
if __name__ == '__main__':
init_db()   # DB + table created automatically
app.run(debug=True)
```

File Name: templates/student.html

```
<!DOCTYPE html>
<html>
<head>
<title>Register</title>
</head>
<body>

<h2>Customer Registration</h2>

<form action="/register" method="POST">
<input type="text" name="name" placeholder="Name" required><br><br>
<input type="email" name="email" placeholder="Email" required><br><br>
<input type="text" name="phone" placeholder="Phone" required><br><br>

<button type="submit">Submit</button>
</form>

</body>
</html>
```

Output 1:

* Serving Flask app 'app'
 * Debug mode: on



PROGRAM 17

Program Name: Write a Python Flask App to create ten static student details list and create an HTML template form which takes input rollno and display the particular student data using GET method.

Steps to install packages in cmd
Pip install flask
Pip install mysql-connector-python
Pip install mysql

Create Folder name like project app:

```
project_app/
│── app.py
│── templates/
   └── register.html
```

To run the program
Cmd –python app.py
Copy web ip address
Paste in browser
Enter input

File Name: app.py

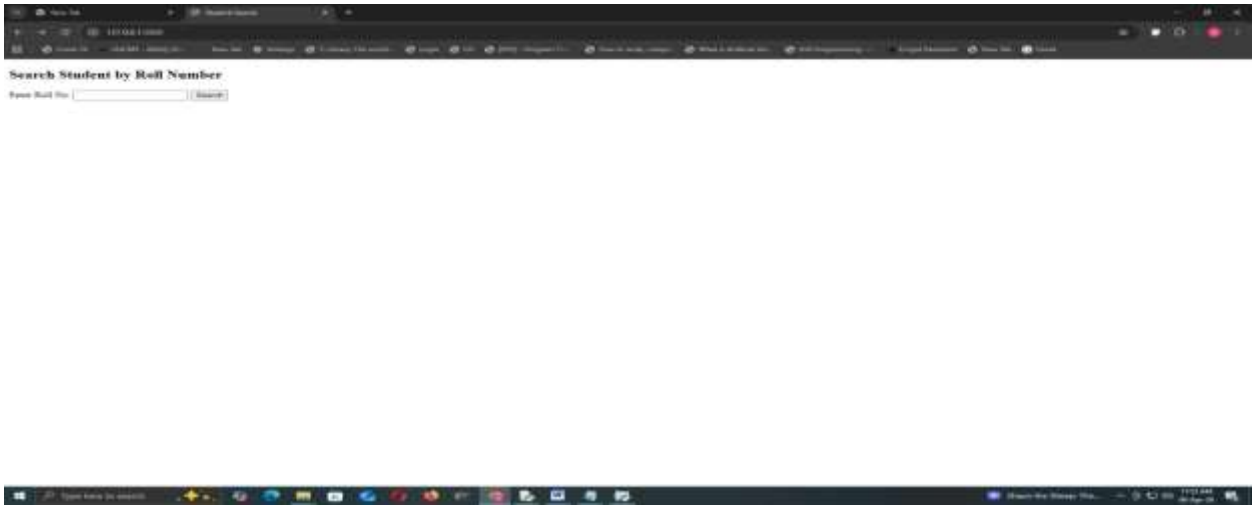
```
from flask import Flask, render_template, request

# Step 1: Create Flask App
app = Flask(__name__)

# Step 2: Static Student Data (10 Records)
students = [
    {"roll": "1", "name": "Amit", "branch": "CSE"},
    {"roll": "2", "name": "Ravi", "branch": "ECE"},
    {"roll": "3", "name": "Sita", "branch": "ME"},
    {"roll": "4", "name": "John", "branch": "CE"},
    {"roll": "5", "name": "Neha", "branch": "EEE"},
    {"roll": "6", "name": "Kiran", "branch": "CSE"},
    {"roll": "7", "name": "Pooja", "branch": "ECE"},
    {"roll": "8", "name": "Rahul", "branch": "ME"},
    {"roll": "9", "name": "Anu", "branch": "CE"},
    {"roll": "10", "name": "Vijay", "branch": "EEE"}
```

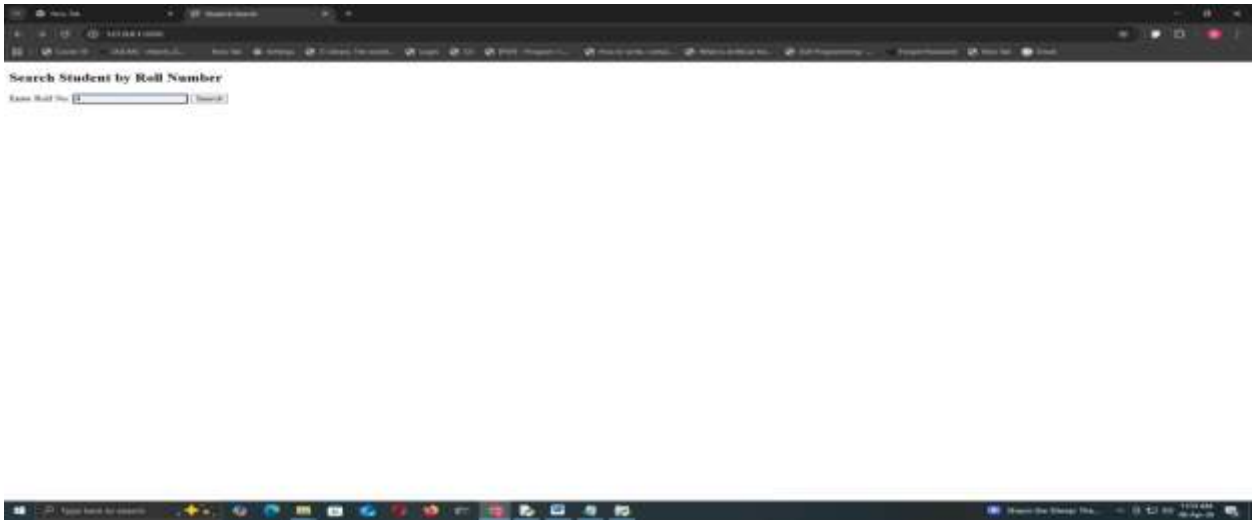
```
]

# Step 3: Route using GET Method
@app.route('/', methods=['GET'])
defsearch_student():

    # Step 4: Get roll number from URL
roll = request.args.get('roll')

student = None

    # Step 5: Search student
if roll:
for s in students:
if s["roll"] == roll:
student = s
break

    # Step 6: Send data to HTML page
returnrender_template('student.html', student=student)

# Step 7: Run the App
if __name__ == '__main__':
app.run(debug=True)
```

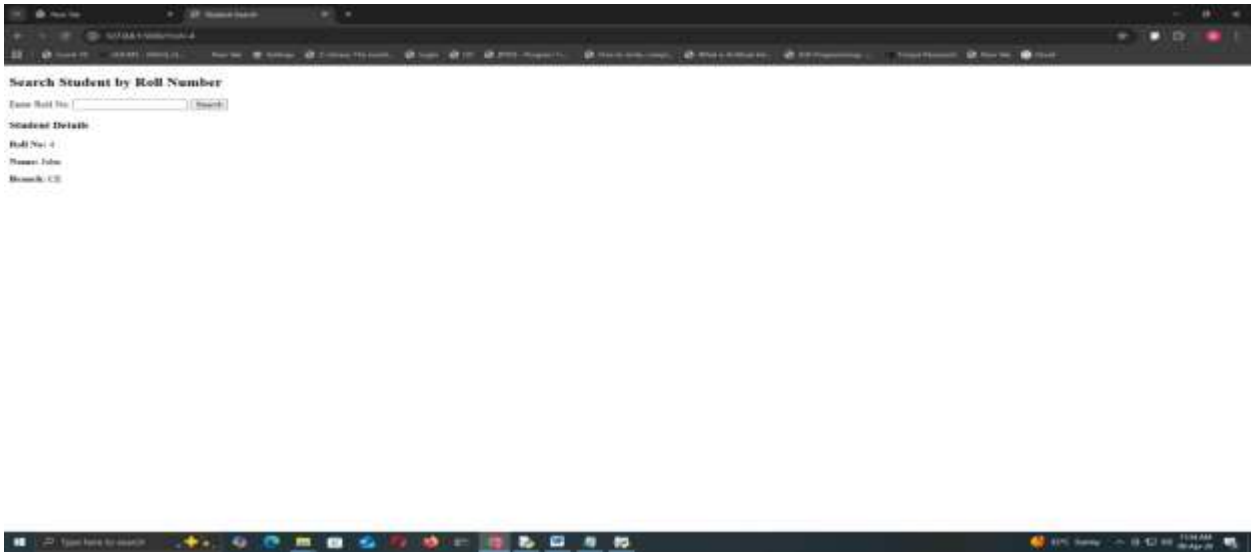
File Name: templates/register.html

```
<!DOCTYPE html>
<html>
<head>
<title>Registration</title>
</head>
<body>
<h2>Student Registration</h2>
<form method="POST">
    Name: <input type="text" name="name" required><br><br>
    Email: <input type="email" name="email" required><br><br>
<input type="submit" value="Register">
</form>
</body>
</html>
```

Output :


Search Student by Roll Number


Search Student by Roll Number


Search Student by Roll Number
Student Details

PROGRAM 18

Program Name: Write a Python program to read product details such as product id, name and price and insert into MySQL database table product and display all products where price greater than 100.

```
import mysql.connector
con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="test")
cur = con.cursor()
pid = int(input("Enter product id: "))
name = input("Enter product name: ")
price = float(input("Enter product price: "))
query = "insert into product values(%s,%s,%s)"
cur.execute(query, (pid, name, price))
con.commit()
print("Products with price greater than 100")
cur.execute("select * from product where price > 100")
for row in cur.fetchall():
    print(row)
con.close()
```

Output 1:
Enter product id: 101
Enter product name: Pen
Enter product price: 120
(101, 'Pen', 120.0)
Output 2:
Enter product id: 102
Enter product name: Book
Enter product price: 80
Only records above 100 displayed

Result:
Thus product details are inserted and retrieved successfully

PROGRAM 19

Write a Python Flask App to create Login HTML Form template, check the credentials in MySQL Database Users table, if exist store in a session and redirect it to another home page route otherwise display error message.

File Name: app.py

```
from flask import Flask, request, render_template, session, redirect
import mysql.connector

app = Flask(__name__)
app.secret_key = 'secretkey'

@app.route('/', methods=['GET', 'POST'])
def login():
   if request.method == 'POST':
uname = request.form['username']
pwd = request.form['password']

       con = mysql.connector.connect(
          host='localhost',
          user='root',
          password='root',
          database='test'
       )

       cur = con.cursor()
       query = "select * from users where username=%s and password=%s"
cur.execute(query, (uname, pwd))
       user = cur.fetchone()
       if user:
          session['user'] = uname
          return redirect('/home')
       else:
          return "Invalid Login"
   return render_template("login.html")
@app.route('/home')
def home():
   return "Welcome " + session['user']
if __name__ == '__main__':
app.run(debug=True)
```

File Name: templates/login.html

```
<html>
<body>
<form method="post">
Username:
<input type="text" name="username"><br><br>
Password:
<input type="password" name="password"><br><br>
<input type="submit" value="Login">
</form>
</body>
</html>
```

SQL Command

```
create table users( username varchar(50), password varchar(50));

insert into users values('admin','123');
```

Output 1:

Username: admin
Password: 123

Welcome admin

Output 2:

Username: user
Password: 111

Invalid Login